Integer Programming and Nonlinear Optimization

Learning Outcomes and Route Through the Lecture

- Choose continuous, integer and binary variables for a production decision.
- Explain an LP relaxation and why rounding can fail.
- Model machine ordering and verify a small schedule with a lower bound.
- Build and trace a knapsack dynamic program.
- Distinguish convex nonlinear optimization from nonconvex problems.

The lecture starts with discrete decisions and ends with nonlinear operating decisions.

Introduction to Integer Programming

- Linear Programming with additional integer constraints
- Some or all decision variables must be integers
- Common in real-world applications where fractional solutions aren't practical
- More complex to solve than linear programming

Types of Integer Programming

- **Pure Integer Programming:**
  - All variables must be integers
  - Example: Number of machines to purchase
- **Mixed Integer Programming:**
  - Some variables are integers, others are continuous
  - Example: Production quantities (integer) and resource usage (continuous)
- **Binary Integer Programming:**
  - Variables can only be 0 or 1
  - Example: Yes/No decisions

An LP Bound Does Not Give a Whole-Machine Plan

Two robot types yield 3 and 2 contribution units per shift:

$$\max 3x + 2y, \qquad 2x + y \leq 4, \quad x + 2y \leq 4, \quad x, y \geq 0.$$

- The LP solution is $(4/3, 4/3)$ with value $20/3$.
- Rounding down gives $(1, 1)$ with value 5.
- The best integer plan is $(2, 0)$ with value 6.
- For maximization, the relaxation gives an upper bound. A feasible integer plan gives a lower bound.

Predict: If we branch on $x \leq 1$ or $x \geq 2$, do these branches cover every integer plan?

Typical IP Application: Scheduling Problems

Scheduling involves assigning limited resources to tasks or time slots over time, respecting various constraints. Integer Programming is often used.

Common Goals:

- Minimize costs (labor, setup, tardiness penalties).
- Complete tasks by deadlines.
- Balance workload across resources.
- Maximize throughput or resource utilization.

Typical Examples:

- Employee shift/roster scheduling.
- Job shop scheduling (assigning jobs to machines).
- Vehicle routing and scheduling.
- Airline crew scheduling.
- Project scheduling (assigning resources to project tasks).

Binary variables ($x_{ij} = 1$ if resource i assigned to task/slot j) are very common.

Example: Employee Weekly Scheduling - Problem

Scenario: Assign 3 employees (Anna, Kate, Peter) to shifts from Monday to Friday to meet daily needs at minimum cost.

Conditions & Constraints:

- **Employees & Costs:** Anna ($150/day), Kate ($160/day), Peter ($140/day).
- **Workload Limit:** Each employee can work max 3 days per week.
- **Daily Needs:** Mon: 1 employee, Tue: 2, Wed: 1, Thu: 1, Fri: 1.
- **Unavailability:**
  - Anna cannot work on Monday.
  - Peter cannot work on Thursday or Friday.

Goal: Minimize total weekly labor cost.

Example: Employee Scheduling - IP Formulation

Let $E = \{\text{Anna, Kate, Peter}\}$, $D = \{\text{Mon, Tue, Wed, Thu, Fri}\}$.

Decision Variables:

- Let $x_{e,d} = 1$ if employee $e \in E$ works on day $d \in D$, and 0 otherwise.

Objective Function (Minimize Cost):

$$\min \sum_{e \in E, d \in D} \text{Cost}_e \cdot x_{e,d}$$

(Where Cost_A=150, Cost_K=160, Cost_P=140, and sum is only over valid e, d pairs based on availability)

Constraints:

- **Max Days per Employee ($\forall e \in E$):** $\sum_{d \in D} x_{e,d} \leq 3$
- **Daily Requirements ($\forall d \in D$):** $\sum_{e \in E} x_{e,d} = \text{Requirement}_d$
  (Requirements: Mon=1, Tue=2, Wed=1, Thu=1, Fri=1)
- **Unavailability:**
  - $x_{\text{Anna, Mon}} = 0$
  - $x_{\text{Peter, Thu}} = 0$
  - $x_{\text{Peter, Fri}} = 0$
- **Variable Type:** $x_{e,d} \in \{0, 1\}$ for all e, d.

Binary Programming Applications

- **Capital Budgeting:**
  - Which projects to invest in
  - Each project is either selected (1) or not (0)
- **Facility Location:**
  - Where to build warehouses
  - Each potential location is either chosen (1) or not (0)
- **Set Covering:**
  - Minimum number of facilities to cover all demand points
  - Each facility is either included (1) or not (0)

Example: Capital Budgeting Problem

Choose projects with a 10 M-dollar budget. All values are teaching data.

Project	Investment (M dollars)	Return (M dollars)
A	3	5
B	4	7
C	2	3
D	5	8
E	1	2

Let $x_i \in \{0, 1\}$ indicate selection.

$$\max 5x_A + 7x_B + 3x_C + 8x_D + 2x_E$$

$$3x_A + 4x_B + 2x_C + 5x_D + x_E \leq 10.$$

A, B, C, E use 10 and return 17. B, D, E also return 17.

Modeling Logical OR Constraints with Binary Variables

Teaching scenario: a cell can use either of two qualified operating modes, each with a different limit on product rates x, y.

Example: Maximize $x + y$ subject to:

- $0 \leq x \leq 10$
- $0 \leq y \leq 10$
- **EITHER** $x + y \leq 3$ (*Constraint 1*)
- **OR** $3y + x \leq 6$ (*Constraint 2*)

Solution using a Binary Variable (d):

- Introduce $d \in \{0, 1\}$.
- If $d = 0$, we want Constraint 1 active ($x + y \leq 3$) and Constraint 2 relaxed.
- If $d = 1$, we want Constraint 2 active ($3y + x \leq 6$) and Constraint 1 relaxed.

The "Big-M" Method

Approach: a binary variable relaxes one mode limit while enforcing the other. Derive each constant from known variable bounds:

- $x + y \leq 3 + M_1 d$
  (Constraint 1: Active if $d = 0$, relaxed if $d = 1$)

- $3y + x \leq 6 + M_2(1 - d)$
  (Constraint 2: Active if $d = 1$, relaxed if $d = 0$)

Choosing Tight Big-M Constants

From $0 \le x, y \le 10$, bound each relaxed left-hand side:

$$M_1 = \max(x + y) - 3 = 20 - 3 = 17,$$
$$M_2 = \max(x + 3y) - 6 = 40 - 6 = 34.$$

- Use $x + y \le 3 + 17d$ and $x + 3y \le 6 + 34(1 - d)$.
- $d = 0$ enforces the first rule. $d = 1$ enforces the second. Both rules may hold.
- A too-small constant cuts out valid plans. Excessively large constants weaken LP bounds and can harm numerical accuracy.

Full Mixed-Integer Formulation

Continuous product rates and a binary mode choice give a mixed-integer linear program:

Maximize:

$$x + y$$

Subject to:

$$0 \leq x \leq 10$$
$$0 \leq y \leq 10$$
$$x + y \leq 3 + 17d$$
$$3y + x \leq 6 + 34(1 - d)$$
$$d \in \{0, 1\}$$

Solving Integer Programming Problems

- **Branch and Bound:**
  - Systematically explores solution space
  - Uses bounds to eliminate suboptimal solutions
- **Cutting Plane Methods:**
  - Adds constraints to eliminate fractional solutions
  - Works with LP relaxation
- **Heuristic Methods:**
  - Greedy algorithms
  - Local search
  - Genetic algorithms

Job Shop Scheduling in a Production Cell

- A job follows a fixed sequence of operations. Different jobs may visit machines in different orders.
- Each operation needs one machine for a known duration. Here operations cannot split or overlap with another operation on that machine.
- Start times and the order of competing operations are decisions.
- The objective is to minimize **makespan**, the completion time of the final job.
- Buffers and waiting are allowed. We ignore setup, transport and machine failures in this teaching model.

A Gantt chart displays the result. The general job-shop problem is NP-hard.

Job Shop Example: 2 Jobs, 2 Machines

Scenario Setup:

- 2 Machines: M0, M1
- 2 Jobs: Job 0, Job 1

Job 0 Details:

- Sequence: M0 $\rightarrow$ M1
- Processing Times: M0 ($p_{0,0} = 5$), M1 ($p_{0,1} = 6$)

Job 1 Details:

- Sequence: M1 $\rightarrow$ M0
- Processing Times: M1 ($p_{1,1} = 4$), M0 ($p_{1,0} = 7$)

Objective: Minimize the makespan (C_{max}).

Job Shop IP Formulation: Variables

Continuous Variables:

- $s_{j,m}$: Start time of Job j on Machine m.
  - $s_{0,0}$, $s_{0,1}$ for Job 0 on M0, M1
  - $s_{1,0}$, $s_{1,1}$ for Job 1 on M0, M1
- C_{max}: The makespan (completion time of the last operation).

Binary Variables (for Machine Sequencing):

- $y_{m,j,k}$: Determines the order of jobs j and k on machine m. Let $y_{m,j,k} = 1$ if j precedes k, 0 if k precedes j.
  - $y_{0,01}$: Order of Job 0 vs Job 1 on Machine 0.
  - $y_{1,01}$: Order of Job 0 vs Job 1 on Machine 1.
- These ensure only one job runs on a machine at any given time.

Each job visits each machine once in this example. For repeated visits, index start times by operation, not only by machine.

Job Shop IP: Objective & Precedence Constraints

Objective Function:

$$\min C_{max}$$

Precedence Constraints (within each job): Ensures operations for a job happen in the correct sequence.

- **Job 0 (M0 $\rightarrow$ M1):** Start on M1 must be after finishing on M0.

$$s_{0,1} \geq s_{0,0} + p_{0,0} \quad \Longrightarrow \quad s_{0,1} \geq s_{0,0} + 5$$

- **Job 1 (M1 $\rightarrow$ M0):** Start on M0 must be after finishing on M1.

$$s_{1,0} \geq s_{1,1} + p_{1,1} \quad \Longrightarrow \quad s_{1,0} \geq s_{1,1} + 4$$

Job Shop IP: Machine Capacity (Big-M)

Ensures no two jobs overlap on the same machine using binary variables ($y_{m,j,k}$) and Big-M.

Machine 0 Constraints (Job 0 vs Job 1):

- If $y_{0,01} = 1$ (Job 0 before Job 1): $s_{1,0} \geq s_{0,0} + p_{0,0}$
  Modeled as: $s_{1,0} \geq s_{0,0} + 5 - M(1 - y_{0,01})$
- If $y_{0,01} = 0$ (Job 1 before Job 0): $s_{0,0} \geq s_{1,0} + p_{1,0}$
  Modeled as: $s_{0,0} \geq s_{1,0} + 7 - My_{0,01}$

Machine 1 Constraints (Job 0 vs Job 1):

- If $y_{1,01} = 1$ (Job 0 before Job 1): $s_{1,1} \geq s_{0,1} + p_{0,1}$
  Modeled as: $s_{1,1} \geq s_{0,1} + 6 - M(1 - y_{1,01})$
- If $y_{1,01} = 0$ (Job 1 before Job 0): $s_{0,1} \geq s_{1,1} + p_{1,1}$
  Modeled as: $s_{0,1} \geq s_{1,1} + 4 - My_{1,01}$

Use the serial schedule horizon $H = 5 + 6 + 4 + 7 = 22$. With $C_{max} \leq 22$ and nonnegative starts, $M = 22$ safely relaxes each alternative.

Job Shop IP: Makespan Definition & Summary

Makespan Constraints: The makespan C_{max} must be greater than or equal to the completion time of the *last operation* of *every* job.

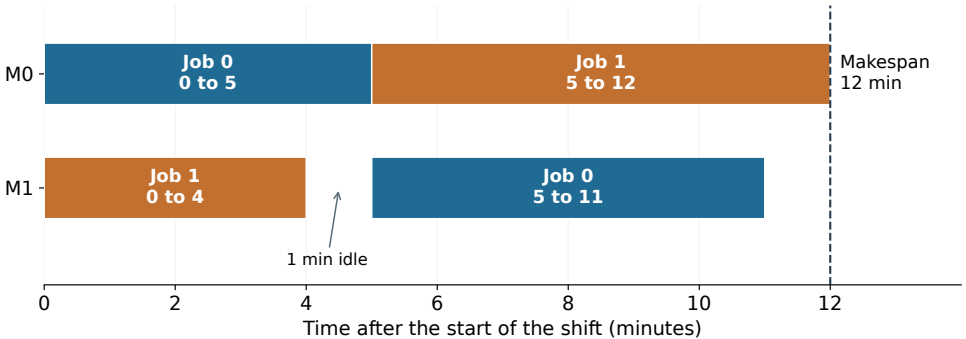
- **Job 0 (ends on M1):** $C_{max} \geq s_{0,1} + p_{0,1} \implies C_{max} \geq s_{0,1} + 6$
- **Job 1 (ends on M0):** $C_{max} \geq s_{1,0} + p_{1,0} \implies C_{max} \geq s_{1,0} + 7$

Complete Formulation Requires:

- Objective: Min C_{max}.
- All Precedence constraints.
- All Machine Capacity (Big-M) constraints.
- All Makespan constraints.
- Horizon and non-negativity: $0 \leq C_{max} \leq 22$, $s_{j,m} \geq 0$.
- Binary variables: $y_{m,j,k} \in \{0, 1\}$.

Solving this Mixed Integer Program yields the optimal schedule and minimum makespan (which is 12 for this example).

A Schedule and a Proof of Optimality



M0 needs $5 + 7 = 12$ minutes. Every schedule therefore has $C_{max} \geq 12$. This feasible schedule attains 12, so it is optimal. **Predict:** Would reducing Job 0's M1 time from 6 to 5 reduce makespan?

Example: Payload Modules for an Inspection Robot

Choose optional modules to maximize inspection score within 10 kg. Each module is available once. Weights and scores are teaching data.

Module	Weight (kg)	Score
1	2	3
2	3	4
3	4	5
4	5	6

Let $x_i = 1$ if module i is included, otherwise $x_i = 0$.

$$\max 3x_1 + 4x_2 + 5x_3 + 6x_4$$

$$2x_1 + 3x_2 + 4x_3 + 5x_4 \leq 10, \qquad x_i \in \{0, 1\}.$$

Predict: Does choosing the highest score per kg always work?

Knapsack DP: Concepts

We solve the Knapsack problem using Dynamic Programming.

DP Formulation Elements:

- **Stages (k):** Decision for item k (considering items $1...k$).
- **State (w):** Available capacity budget for selecting from the first k items.
- **Value Function ($f_k(w)$):** Max value using items $1...k$ with capacity *at most* w.

The goal is to find $f_n(W)$, where n is the number of items and W is the total capacity.

Knapsack DP: Recursive Relation

To compute $f_k(w)$, we decide about item k (weight w_k, value v_k):

- **If item k is NOT included:** Value is $f_{k-1}(w)$.
- **If item k IS included (requires $w \geq w_k$):** Value is $v_k + f_{k-1}(w - w_k)$.

We take the maximum of the possible options:

$$f_k(w) = \begin{cases} f_{k-1}(w) & \text{if } w < w_k \\ \max\{f_{k-1}(w),\ \ v_k + f_{k-1}(w - w_k)\} & \text{if } w \geq w_k \end{cases}$$

Base case: $f_0(w) = 0$ for all w.

Knapsack DP - Table Calculation (w=0 to 5)

Compute $f_k(w)$ row by row. W=10. Items: 1:(2,3), 2:(3,4), 3:(4,5), 4:(5,6).

k (Item)	w=0	w=1	w=2	w=3	w=4	w=5
0 (f_0)	0	0	0	0	0	0
1 (2,3) (f_1)	0	0	3	3	3	3
2 (3,4) (f_2)	0	0	3	4	4	7
3 (4,5) (f_3)	0	0	3	4	5	7
4 (5,6) (f_4)	0	0	3	4	5	7

(Table continues on next slide...)

Knapsack DP - Table Calculation (w=6 to 10)

Continuing the table calculation...

k (Item)	w=6	w=7	w=8	w=9	w=10
0 (f_0)	0	0	0	0	0
1 (2,3) (f_1)	3	3	3	3	3
2 (3,4) (f_2)	7	7	7	7	7
3 (4,5) (f_3)	8	9	9	12	12
4 (5,6) (f_4)	8	9	10	12	**13**

The maximum value considering all items and capacity W=10 is $f_4(10) = 13$.

Knapsack DP - Finding the Solution

Maximum value from table: $f_4(10) = \mathbf{13}$.

Backtracking to find items:

- $f_4(10) = 13 \neq f_3(10) = 12 \implies$ Item 4 (5,6) **is included**.
- Check remaining value $13 - 6 = 7$ at capacity $10 - 5 = 5$: $f_3(5)$.
- $f_3(5) = 7 = f_2(5) = 7 \implies$ Item 3 (4,5) is **not** included.
- Check value 7 at capacity 5: $f_2(5)$.
- $f_2(5) = 7 \neq f_1(5) = 3 \implies$ Item 2 (3,4) **is included**.
- Check remaining value $7 - 4 = 3$ at capacity $5 - 3 = 2$: $f_1(2)$.
- $f_1(2) = 3 \neq f_0(2) = 0 \implies$ Item 1 (2,3) **is included**.

Solution: Include Items **1, 2, 4**.

- Total Weight $= 2 + 3 + 5 = 10$ kg
- Total Value $= 3 + 4 + 6 = 13$

Why the Knapsack Recurrence Is Useful

- Interpret items as optional inspection modules for a mobile robot with a 10 kg payload budget. Values are illustrative inspection scores.
- At $(k, w) = (4, 10)$, excluding item 4 gives 12. Including it gives $6 + f_3(5) = 13$.
- Each transition reads the previous row, so every module can appear at most once.
- The table uses $O(nW)$ time and space for integer weights and capacity. It is pseudopolynomial: the number of columns grows with the numeric capacity.

Check: Capacity 9 gives score 12 using items 1, 2 and 3. One more kg changes the best combination.

Nonlinear Optimization

- Objective function or constraints are nonlinear
- More complex than linear programming
- Common types:
  - Quadratic Programming
  - Convex Optimization
  - Non-convex Optimization

Nonlinear Relationships in Production

- **Price depends on quantity:** a price $a - bx$ gives revenue $ax - bx^2$.
- **Energy depends on speed:** energy per cycle may rise rapidly at high robot speeds.
- **Yield depends on operating settings:** temperature or force can change defect rates nonlinearly.
- **Scale changes costs:** overtime or quantity discounts may require nonlinear or piecewise-linear models.

The form matters. Convex minimization and concave maximization allow global guarantees. General nonconvex models require more care.

NLP Example: Quantity-Dependent Price

Assume selling more requires a lower unit price:

$$P(x) = a - bx, \qquad R(x) = P(x)x = ax - bx^2, \quad b > 0.$$

For cost $C(x) = cx + d$, profit is

$$\Pi(x) = (a - c)x - bx^2 - d.$$

- Choose a quantity x with $0 \leq x \leq K$ and nonnegative price.
- Price, revenue and profit have different units. Multiplying unit price by quantity creates the quadratic term.
- This downward-sloping inverse demand curve does not imply elastic demand at every quantity.

Another Quadratic Model: Portfolio Risk

Let x_i be the fraction invested in asset i, r_i its expected return, and $\Sigma = (\sigma_{ij})$ the covariance matrix of returns.

$$\min_x x^T \Sigma x$$

$$\sum_i r_i x_i \geq R, \qquad \sum_i x_i = 1, \qquad x_i \geq 0.$$

- R is the required portfolio return. The objective measures variance.
- A covariance matrix is positive semidefinite, so this is a convex quadratic program.
- Expected returns and covariances are estimates. An optimization guarantee concerns the model, not future realized performance.

Methods for Nonlinear Optimization

- **Gradient Methods:**
  - Steepest descent
  - Newton's method
  - Conjugate gradient
- **Interior Point Methods:**
  - Barrier methods
  - Primal-dual methods
- **Global Optimization:**
  - Branch and bound
  - Simulated annealing
  - Genetic algorithms

A Nonlinear Production Decision With a Global Answer

For a teaching example, let price $P(x) = 100 - 2x$, cost $C(x) = 20x + 100$, and $0 \le x \le 30$ units per shift.

$$\Pi(x) = 80x - 2x^2 - 100, \qquad \Pi'(x) = 80 - 4x.$$

- The stationary point is $x = 20$. Profit is 700 contribution units per shift.
- Since $\Pi''(x) = -4 < 0$, profit is strictly concave. This feasible stationary point is the unique global maximum.
- A capacity of 15 moves the optimum to the boundary $x = 15$, with profit 650.
- Nonconvex models may have several local optima. A local solver alone does not certify a global answer.

Key Takeaways

- Integer programming adds complexity to optimization
- Binary variables enable modeling of yes/no decisions
- Nonlinear optimization handles more complex relationships
- Dynamic programming is powerful for certain problem types
- Choice of method depends on problem characteristics

Whole-robot choices in PuLP

```
model = pulp.LpProblem("Robot_mix", pulp.LpMaximize)
x = pulp.LpVariable("type_x", lowBound=0, cat=pulp.LpInteger)
y = pulp.LpVariable("type_y", lowBound=0, cat=pulp.LpInteger)
model += 3*x + 2*y, "contribution"
model += 2*x + y <= 4, "assembly_capacity"
model += x + 2*y <= 4, "testing_capacity"
model.solve(pulp.PULP_CBC_CMD(msg=False))
```

Using `LpContinuous` instead gives the LP bound. The integer optimum is $(2, 0)$ with contribution 6. The scheduling companion also uses `LpBinary` to choose each machine's order, with the lecture's horizon-based Big-M constraints. It checks the result against all four fixed-order cases.

Lecture Companion and Next Lab

- `notebooks/lecture_2_discrete_optimization.ipynb`: a 20-minute demonstration with PuLP 3.x/CBC.
- Compare an LP bound with integer robot choices, trace one knapsack decision, and test the two-machine schedule.
- Lab 3 develops integer cutting patterns. Lab 4 scales scheduling to several kits and stations.

Exit questions: What proves a schedule optimal? Why does reducing one operation's duration sometimes leave makespan unchanged?